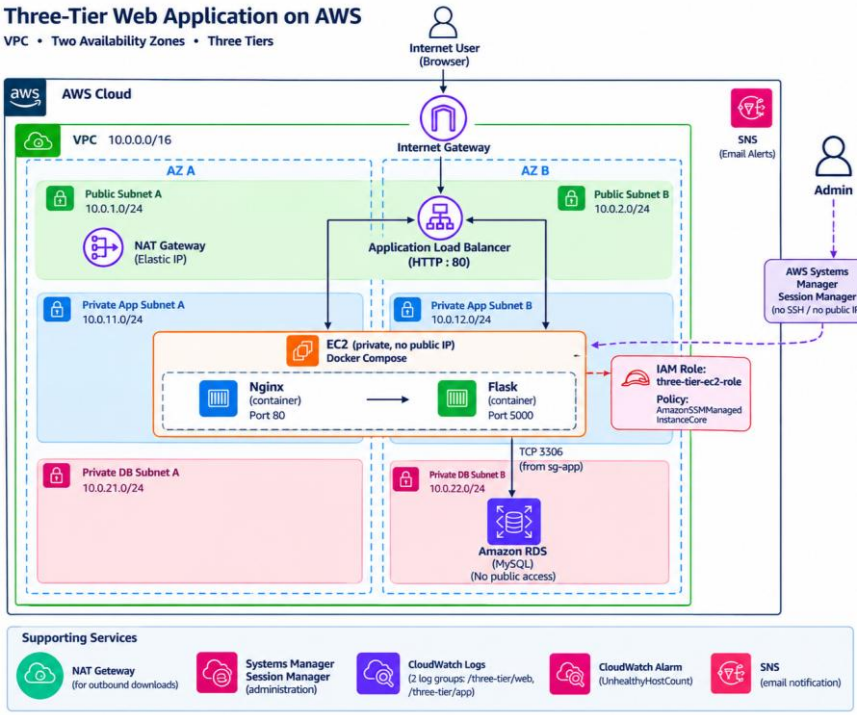
AWS THREE TIER APPLICATION PROJECT TASK

ARCHITECTURE DIAGRAM:



CREATION OF VPC AND 6 SUBNETS:

1)navigate to vpc service in your aws management console

2)click create vpc and select vpc only.

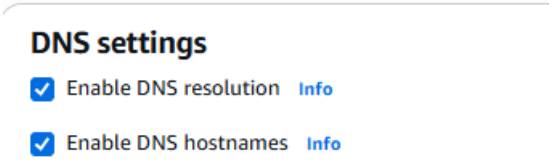
3)give a name to your vpc as (three-tier-vpc)

4)choose ipv4 CIDR manual input.

5)ipv4 CIDR 10.0.0.0/16

6)keep tenancy as default and click create vpc.

7)once the vpc is created select your vpc and click actions and select vpc settings. Enable the DNS resolution and DNS hostnames



CREATION OF SUBNET: (create 3 subnet in one availability zone and another three in another availability zone)

1)navigate to subnet section in vpc service.

2)click create subnet **(public-a)**

3)select the created vpc (three-tier-vpc)

4)choose the availability zone as **Asia Pacific (Mumbai)/aps1-az1(ap-south-1a)**

5)ipv4 vpc CIDR block 10.0.0.0/16 and ipv4 subnet CIDR block 10.0.1.0/24

6)click create subnet.

7)click create subnet **(public-b)**

8)select the created vpc (three-tier-vpc)

9)choose the availability zone as **Asia Pacific (Mumbai)/aps1-az3(ap-south-1b)**

10)ipv4 vpc CIDR block 10.0.0.0/16 and ipv4 subnet CIDR block 10.0.2.0/24

11)click create subnet.

12)click create subnet **(app-a)**

13)select the created vpc (three-tier-vpc)

14)choose the availability zone as **Asia Pacific (Mumbai)/aps1-az1(ap-south-1a)**

15)ipv4 vpc CIDR block 10.0.0.0/16 and ipv4 subnet CIDR block 10.0.11.0/24

16)click create subnet.

17)click create subnet **(app-b)**

18)select the created vpc (three-tier-vpc)

19)choose the availability zone as **Asia Pacific (Mumbai)/aps1-az3(ap-south-1b)**

20)ipv4 vpc CIDR block 10.0.0.0/16 and ipv4 subnet CIDR block 10.0.12.0/24

21)click create subnet.

22)click create subnet **(db-a)**

23)select the created vpc (three-tier-vpc)

24)choose the availability zone as **Asia Pacific (Mumbai)/aps1-az1(ap-south-1a)**

25)ipv4 vpc CIDR block 10.0.0.0/16 and ipv4 subnet CIDR block 10.0.21.0/24

26)click create subnet.

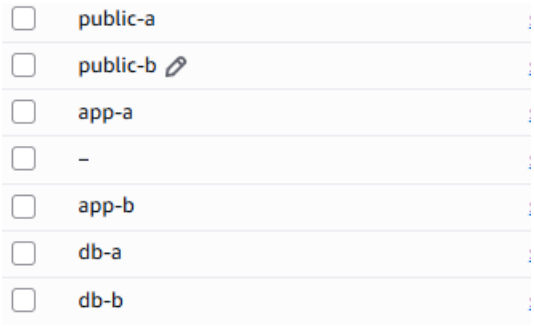
27)click create subnet **(db-b)**

28)select the created vpc (three-tier-vpc)

29)choose the availability zone as **Asia Pacific (Mumbai)/aps1-az3(ap-south-1b)**

30)ipv4 vpc CIDR block 10.0.0.0/16 and ipv4 subnet CIDR block 10.0.21.0/24

31)click create subnet



CREATE AND ATTACH AN INTERNET GATEWAY:

1) navigate to internet gateway under vpc service(left side of console)

2)click create internet gateway. And give name (three-tier-igw)

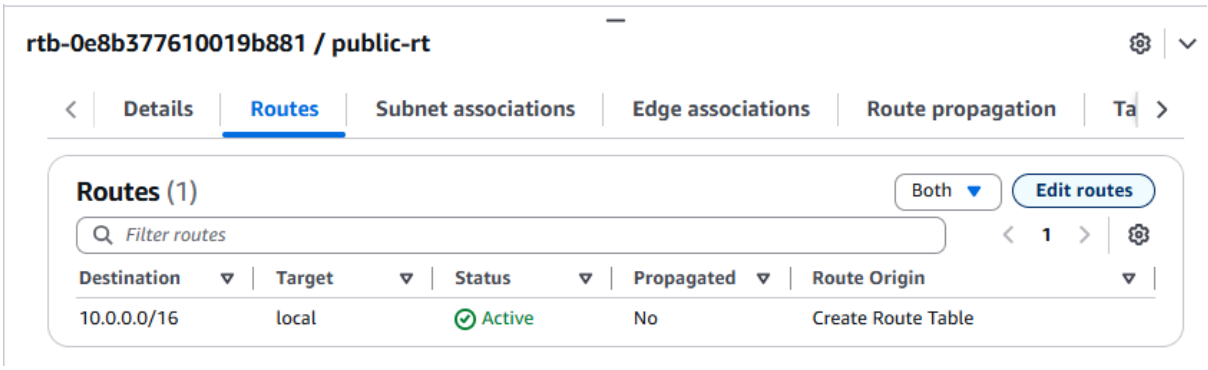
3)after the creation click attach to vpc.

4)choose three-tier-vpc and click attach.

CREATE THE PUBLIC ROUTE TABLE:

1) navigate to route table under vpc service(left side of console)

2)click create route table and give a name to your route table(public-rt)



3)select the created vpc. And click create route table.

4)once the route table created navigate to routes tab and click edit routes

5)click add routes and choose 0.0.0.0.0/0 and choose igw click save changes

6)and switch to subnet association and associate with **public-a** and **public-b** with the public-rt

7)click save changes.

CREATE THE NAT GATEWAY:

1) navigate to NAT gateway under vpc service(left side of console)

2)click create nat gateway and give a name **(three-tier-nat).**

3)choose zonal as availability mode and choose the public-a subnet

4)connectivity type as public click allocate elastic ip

5)click create nat gateway.

CREATE THE 2 PRIVATE ROUTE TABLES:

1) navigate to route table under vpc service(left side of console)

2)click create route table and give a name to your route table(app-private-rt)

3)select the created vpc. And click create route table.

4)once the route table created navigate to routes tab and click edit routes

5)click add routes and choose 0.0.0.0.0/0 and choose NAT click save changes

6)switch to subnet association and associate with **app-a** and **app-b** with the app-private-rt

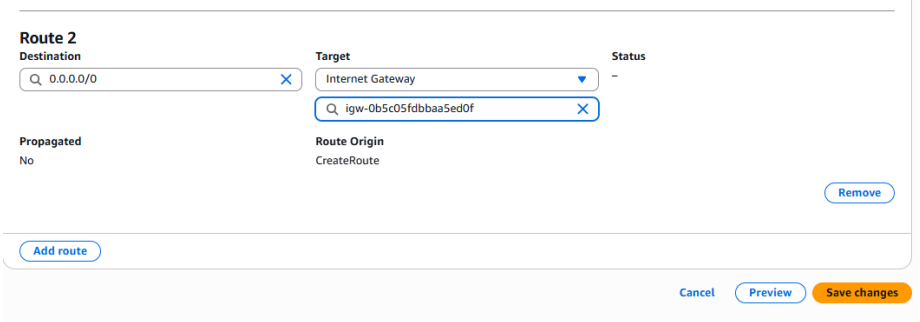
7)click create route table and give a name to your route table(db-private-rt)

8)select the created vpc. And click create route table.

9)switch to subnet association and associate with **db-a** and **db-b** with the db-private-rt

CREATE THE 3 SECURITY GROUPS using created vpc:

1)navigate to ec2 service and navigate security group



2)give a name (alb-sg)

3)select the vpc and in inbound rules select http and give anywhereipv4

4)click create security group (app-sg)

5) Inbound rule as Type: Custom TCP, Port: 80, Source: Security group alb-sg

6)click create security group (db-sg)

5) Inbound rule as Type: mysql/aurora, Port: 3306, Source: Security group app-sg

CREATE RDS SUBNET GROUP:

1)navigate to rds service and click subnet group click create.

2)give a name to your subnet group(three-tier-db-subnet-group)

3)select the vpc and 2 availability zone.

4)select db-a,db-b subnet click create.

CREATE RDS:

1)choose engine type as my sql.

2)choose full configuration.

3)choose dev/test and select Single-AZ DB instance deployment (1 instance)

4)choose my sql community and version 8.4.9

5)give a identifier name three-tier-mysql

6)give the master username as dbadmin and pass: dbadmin1234

7)select password authentication

8)choose instance configuration as burstable class and instance type as db.t3.micro.

9)storage type as general purpose

10)under connectivity don’t connect to ec2

11)network type as ipv4.

12) select three-tier vpc

13)select the created three tier db subnet group.

14)no public access.

15)choose vpc security group existing db-sg and availability zone as no preference and uncheck the rds proxy

16)click create database

CREATE THE IAM ROLE FOR EC2:

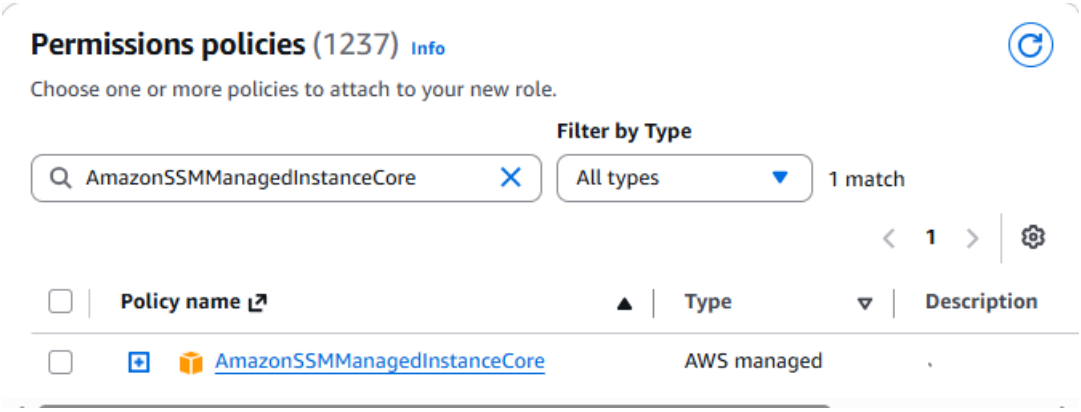
1)navigate to iam service.

2)click roles and click create roles.

3)choose the entity type as aws service.

4)choose ec2 as system manager.click next

5)choose permission policy as AmazonSSMManagedInstanceCore



6)give a name to your role (three-tier-ec2-role.)

7)click create.

CREATE A PRIVATE EC2 INSTANCE:

1)navigate to ec2 service and click launch instance and give a name to your instance (three-tier-app-server)

2)choose amazon ami as linux 2023

3)choose three tier vpc and subnet as app-a

4)disable autoassign public ip select app-sg as security group.

5)in additional settings click iam instance profile as three tier ec2 role.

6)click create launch instance.

(before connecting attach the iam role)

CONNECT TO THE INSTANCE:

(It may take few minutes navigate to session manager service and choose your instance and click start session)

```
sudo dnf update -y
sudo dnf install -y docker git
sudo systemctl enable --now docker
docker --version
git --version
```

sudo curl -fSL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 -o /usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose

git clone --depth 1 https://github.com/docker/awesome-compose.git

cd awesome-compose/nginx-flask-mysql

ls -la

```
[root@ip-10-0-11-48 ~]# sudo curl -fSL https://github.com/docker/compose/releases/latest/download/docke
-compose-linux-x86_64 -o /usr/local/bin/docker-compose
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0      0   0      0   0      0      0      0                              0
  0      0   0      0   0      0      0      0                              0
100 31.14M 100 31.14M   0      0 48.24M      0                              0
[root@ip-10-0-11-48 ~]# sudo chmod +x /usr/local/bin/docker-compose
[root@ip-10-0-11-48 ~]# docker-compose version
Docker Compose version v5.6.0
[root@ip-10-0-11-48 ~]# git clone --depth 1 https://github.com/docker/awesome-compose.git
Cloning into 'awesome-compose'...
remote: Enumerating objects: 606, done.
remote: Counting objects: 100% (606/606), done.
remote: Compressing objects: 100% (465/465), done.
remote: Total 606 (delta 71), reused 468 (delta 54), pack-reused 0 (from 0)
Receiving objects: 100% (606/606), 3.73 MiB | 26.87 MiB/s, done.
Resolving deltas: 100% (71/71), done.
[root@ip-10-0-11-48 ~]# cd awesome-compose/nginx-flask-mysql
[root@ip-10-0-11-48 nginx-flask-mysql]# ls -la
total 24
drwxr-xr-x.  5 root root    81 Oct 16 07:53 .
drwxr-xr-x. 44 root root 16384 Oct 16 07:53 ..
-rw-r--r--.  1 root root  2562 Oct 16 07:53 README.md
drwxr-xr-x.  2 root root    64 Oct 16 07:53 backend
-rw-r--r--.  1 root root  1239 Oct 16 07:53 compose.yaml
drwxr-xr-x.  2 root root    26 Oct 16 07:53 db
drwxr-xr-x.  2 root root    36 Oct 16 07:53 proxy
[root@ip-10-0-11-48 nginx-flask-mysql]#
```

cp compose.yaml compose.yaml.backup

pwd

ls

grep -RniE 'MYSQL|MARIADB|DB_HOST|DB_PASSWORD|pymysql|mysql.connector|sqlalchemy' backend

cat backend/hello.py

```
cp backend/hello.py backend/hello.py.backup
cat > backend/hello.py <<'EOF'
import os
from flask import Flask
import mysql.connector

class DBManager:
    def __init__(self):
        self.connection = mysql.connector.connect(
            host=os.environ["DB_HOST"],
            port=int(os.getenv("DB_PORT", "3306")),
            user=os.environ["DB_USER"],
            password=os.environ["DB_PASSWORD"],
            database=os.environ["DB_NAME"]
        )
        self.cursor = self.connection.cursor()

    def populate_db(self):
        self.cursor.execute(
            'CREATE TABLE IF NOT EXISTS blog '
            '(id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255))'
        )
        self.cursor.execute('SELECT COUNT(*) FROM blog')
        count = self.cursor.fetchone()[0]

        if count == 0:
            self.cursor.executemany(
                'INSERT INTO blog (title) VALUES (%s)',
```

```
            [(f'Blog post #{i}',) for i in range(1, 5)]
        )
        self.connection.commit()

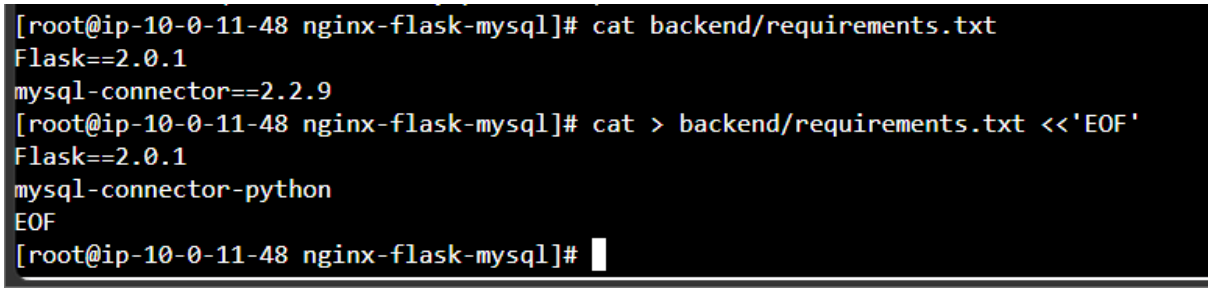
    def query_titles(self):
        self.cursor.execute('SELECT title FROM blog')
        return [row[0] for row in self.cursor.fetchall()]


server = Flask(__name__)
conn = None

@server.route('/')
def listBlog():
    global conn
    if conn is None:
        conn = DBManager()
        conn.populate_db()

    rec = conn.query_titles()
    return ''.join(f'<div>Hello {title}</div>' for title in rec)

if __name__ == '__main__':
    server.run(host='0.0.0.0', port=8000)
EOF
cat backend/hello.py
```



cat backend/requirements.txt

ls -R proxy

ls -R proxy/conf

cat proxy/Dockerfile

proxy/conf/default.conf <<'EOF'

server {

    listen 80;

    server_name _;

    location / {

        proxy_pass http://backend:8000;

        proxy_set_header Host $host;

        proxy_set_header X-Real-IP $remote_addr;

        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_set_header X-Forwarded-Proto $scheme;

    }

}

EOF

-bash: proxy/conf/default.conf: Not a directory

[root@ip-10-0-11-48 nginx-flask-mysql]# cat proxy/conf/default.conf

cat: proxy/conf/default.conf: Not a directory

[root@ip-10-0-11-48 nginx-flask-mysql]# cat > proxy/conf <<'EOF'

server {

```
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
[root@ip-10-0-11-48 nginx-flask-mysql]# cat proxy/conf
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
cat > compose.yaml <<'EOF'
services:
  backend:
```

```
    build:
      context: backend
      target: builder
    restart: always
    env_file:
      - .env
    expose:
      - "8000"
    networks:
      - frontnet

  proxy:
    build: proxy
    restart: always
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - frontnet

networks:
  frontnet:
EOF
[root@ip-10-0-11-48 nginx-flask-mysql]# cat compose.yaml
services:
  backend:
    build:
```

```
    context: backend
    target: builder
  restart: always
  env_file:
    - .env
  expose:
    - "8000"
  networks:
    - frontnet

 proxy:
  build: proxy
  restart: always
  ports:
    - "80:80"
  depends_on:
    - backend
  networks:
    - frontnet

networks:
 frontnet:
[root@ip-10-0-11-48 nginx-flask-mysql]# cat backend/Dockerfile
# syntax=docker/dockerfile:1.4
FROM --platform=$BUILDPLATFORM python:3.10-alpine AS builder

WORKDIR /code
COPY requirements.txt /code
```

```
RUN --mount=type=cache,target=/root/.cache/pip \
    pip3 install -r requirements.txt

COPY . .

ENV FLASK_APP hello.py
ENV FLASK_ENV development
ENV FLASK_RUN_PORT 8000
ENV FLASK_RUN_HOST 0.0.0.0

EXPOSE 8000

CMD ["flask", "run"]

FROM builder AS dev-envs

RUN <<EOF
apk update
apk add git
EOF

RUN <<EOF
addgroup -S docker
adduser -S --shell /bin/bash --ingroup docker vscode
EOF

# install Docker tools (cli, buildx, compose)
COPY --from=gloursdocker/docker / /
```

CMD ["flask", "run"]

Vi env

DB_HOST= three-tier-mysql.cbsu2mgqemmh.ap-south-1.rds.amazonaws.com

DB_PORT=3306

DB_NAME= mysql

DB_USER= dbadmin

DB_PASSWORD= dbadmin1234

```
[root@ip-10-0-11-48 nginx-flask-mysql]# docker-compose version
ls -l .env
Docker Compose version v5.6.0
-rw-------. 1 root root 139 Oct 16 08:14 .env
[root@ip-10-0-11-48 nginx-flask-mysql]# docker-compose config --quiet
[root@ip-10-0-11-48 nginx-flask-mysql]# grep -E '^(DB_HOST|DB_PORT|DB_NAME|DB_USER)=' .env
DB_HOST= three-tier-mysql.cbsu2mgqemmh.ap-south-1.rds.amazonaws.com
DB_PORT=3306
DB_NAME= mysql
DB_USER= dbadmin
```

docker-compose config –quiet

grep -E '^(DB_HOST|DB_PORT|DB_NAME|DB_USER)=' .env

timeout 5 bash -c '</dev/tcp/three-tier-mysql.cbsu2mgqemmh.ap-south-1.rds.amazonaws.com/3306' \

&& echo "RDS port reachable" \

|| echo "RDS port not reachable"

```
[root@ip-10-0-11-48 nginx-flask-mysql]# timeout 5 bash -c '</dev/tcp/three-tier-mysql.cbsu2mgqemmh.ap-south-1.rds.amazonaws.com/3306' \
  && echo "RDS port reachable" \
  || echo "RDS port not reachable"
RDS port reachable
```

dnf install -y mariadb105

mysql --version

mysql -h three-tier-mysql.cbsu2mgqemmh.ap-south-1.rds.amazonaws.com \

-P 3306 -u dbadmin -p

SHOW DATABASES;

CREATE DATABASE example;

SHOW DATABASES;

USE example;

EXIT;

sed -i 's/^DB_NAME=.*/DB_NAME=example/' .env

```
ySQL [(none)]> CREATE DATABASE example;
uery OK, 1 row affected (0.027 sec)

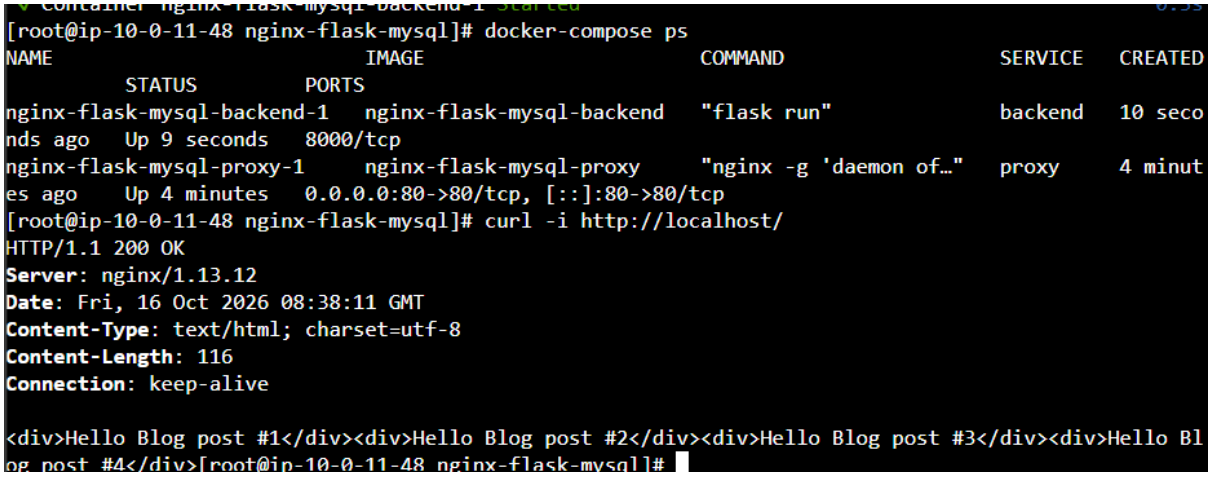
ySQL [(none)]> SHOW DATABASES;
--------------------+
 Database           |
--------------------+
 example            |
 information_schema |
 mysql              |
 performance_schema |
 sys                |
--------------------+
 rows in set (0.007 sec)

ySQL [(none)]> USE example;
atabase changed
ySQL [example]>
```

```
[root@ip-10-0-11-48 nginx-flask-mysql]# sed -i 's/^DB_NAME=.*/DB_NAME=example/' .env
[root@ip-10-0-11-48 nginx-flask-mysql]# grep '^DB_NAME=' .env
DB_NAME=example
[root@ip-10-0-11-48 nginx-flask-mysql]#
```

docker-compose up -d –build

```
[+] up 5/5
 ✓ Image nginx-flask-mysql-backend       Built                                12.4s
 ✓ Image nginx-flask-mysql-proxy         Built                                12.4s
 ✓ Network nginx-flask-mysql_frontnet    Created                               0.2s
 ✓ Container nginx-flask-mysql-backend-1 Started                               0.6s
 ✓ Container nginx-flask-mysql-proxy-1   Started                               1.0s
```



Now I have completed the Client / Browser>>Nginx reverse proxy>>Flask backend Port 8000 · Docker container>>Amazon RDS MySQLPort 3306 · Private database

Create a target group:

1)choose instance.

2)give a name(three-tier-tg)

3)protocol hhtp port 80 adress type ipv4

4)select vpc click next choose the instance and click include as pending below click create target group

Create and connect the application load balancer:

1)navigate to load balancer.

2)click create load balancer.choose alb

3)choose the scheme as internet facing.

4)choose ipv4.

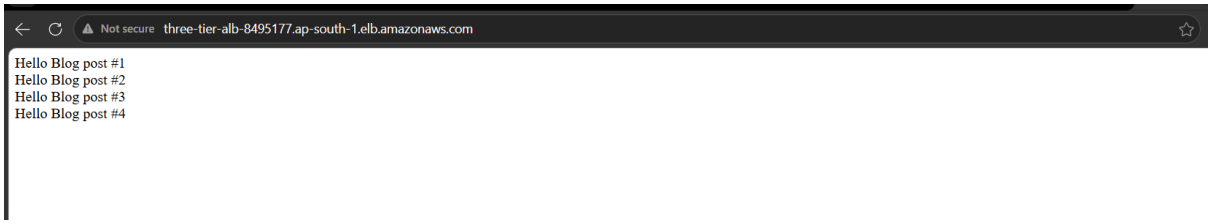
5)select the created vpc.

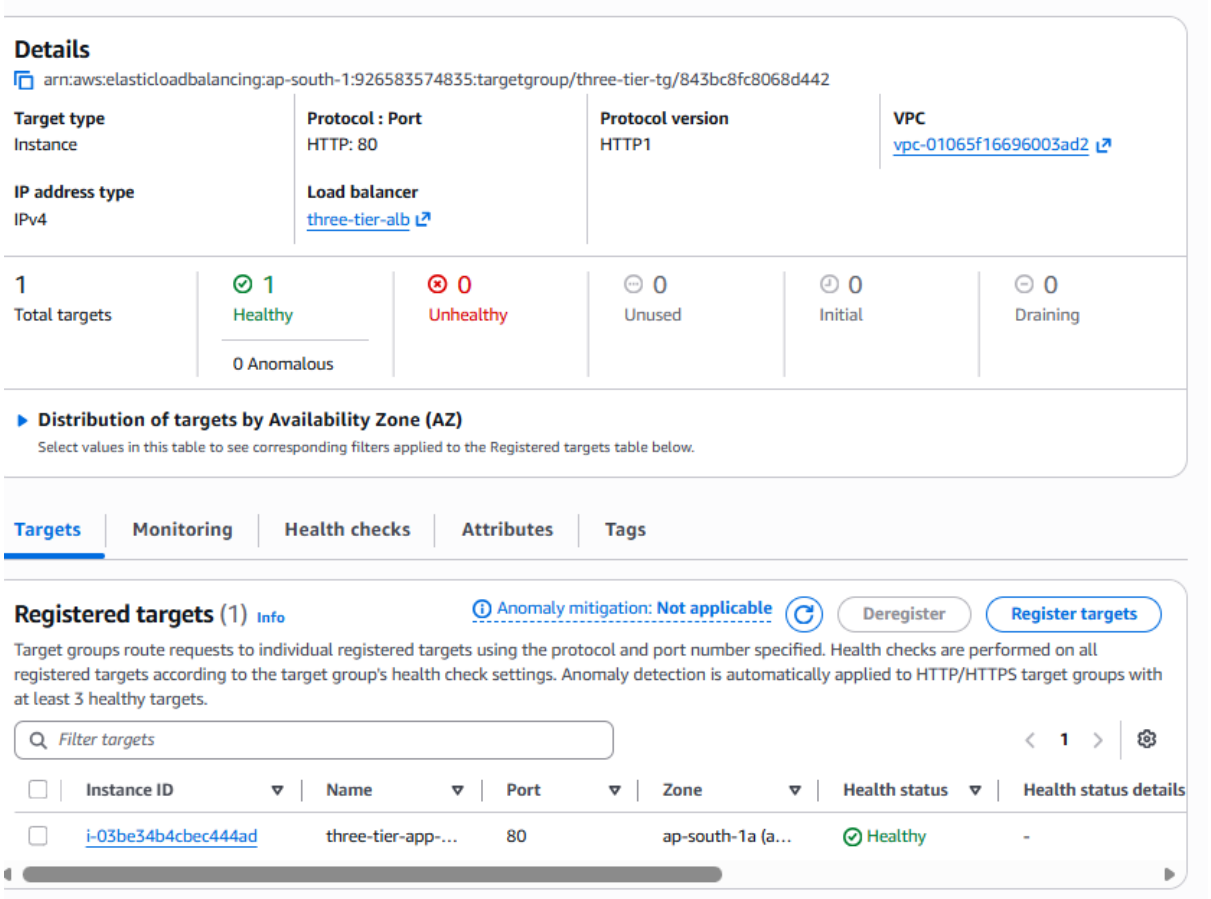
6)select the target group.

7)click create load balancer.

Output:

(one the load balance status turned to active copy the dns and paste in new tab)





docker inspect nginx-flask-mysql-backend-1 --format '{{.Name}} restart={{.HostConfig.RestartPolicy.Name}}'

docker inspect nginx-flask-mysql-proxy-1 --format '{{.Name}} restart={{.HostConfig.RestartPolicy.Name}}'

curl http://localhost/

the 4 blogpost is displayed



```
Return low-level information on Docker objects
[root@ip-10-0-11-48 nginx-flask-mysql]# docker inspect nginx-flask-mysql-backend-1 --format '{{.Name}} restart={{.HostConfig.RestartPolicy.Name}}'
/nginx-flask-mysql-backend-1 restart=always
[root@ip-10-0-11-48 nginx-flask-mysql]# docker inspect nginx-flask-mysql-proxy-1 --format '{{.Name}} restart={{.HostConfig.RestartPolicy.Name}}'
/nginx-flask-mysql-proxy-1 restart=always
[root@ip-10-0-11-48 nginx-flask-mysql]#
```

cd /root/awesome-compose/nginx-flask-mysql

ls -l .env

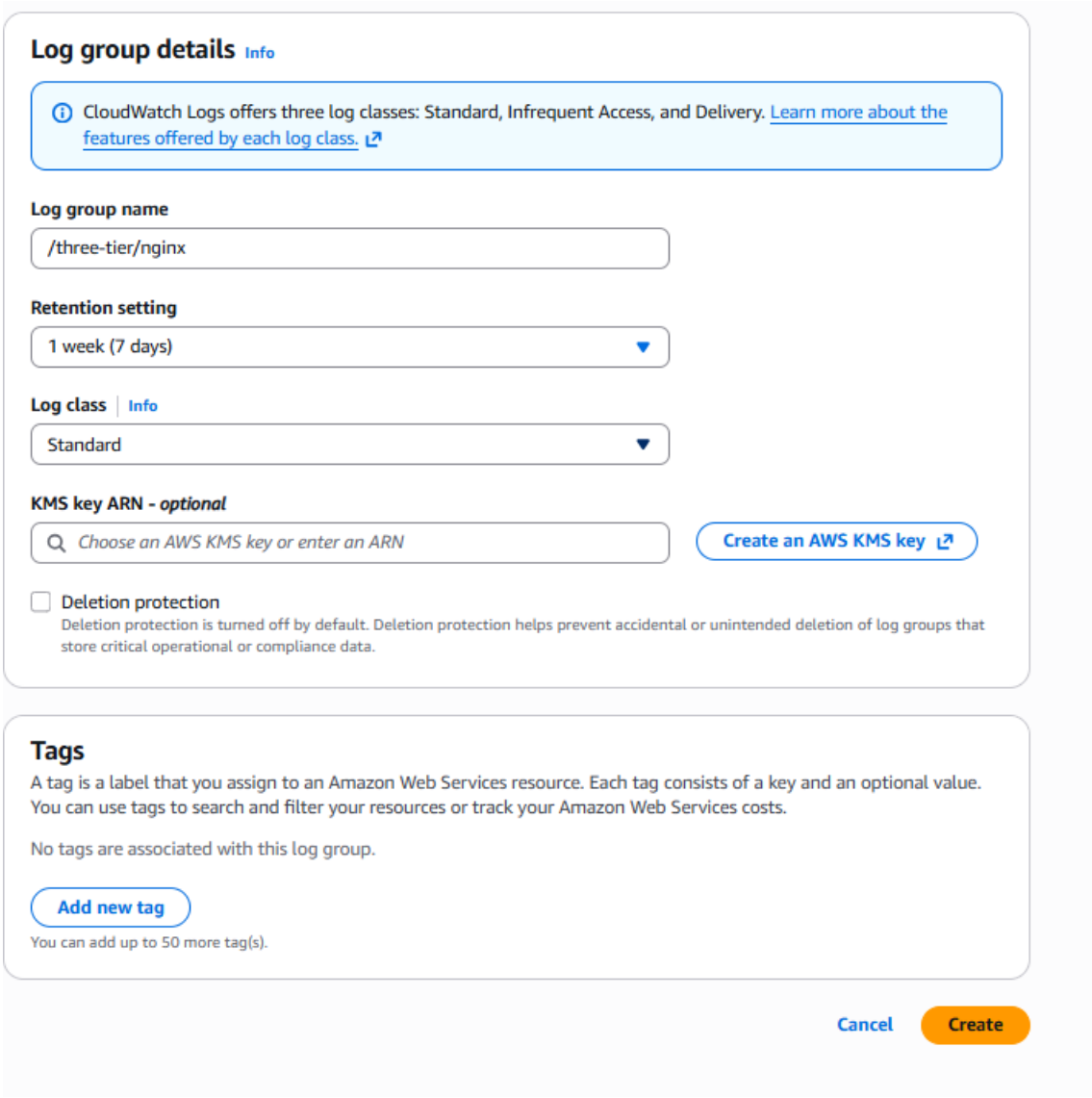
```
[root@ip-10-0-11-48 ~]# cd /root/awesome-compose/nginx-flask-mysql
[root@ip-10-0-11-48 nginx-flask-mysql]# ls -l .env
-rw-------. 1 root root 137 Oct 16 08:29 .env
[root@ip-10-0-11-48 nginx-flask-mysql]#
```

this shows the database is secured

CREATE CLOUD WATCH:

1)navigate to the cloud watch service and click logs and click create log groups.

2)give a name and give retention period as one week and click create.



3)create another cloud watch with a name /three-tier/flask with same retention.

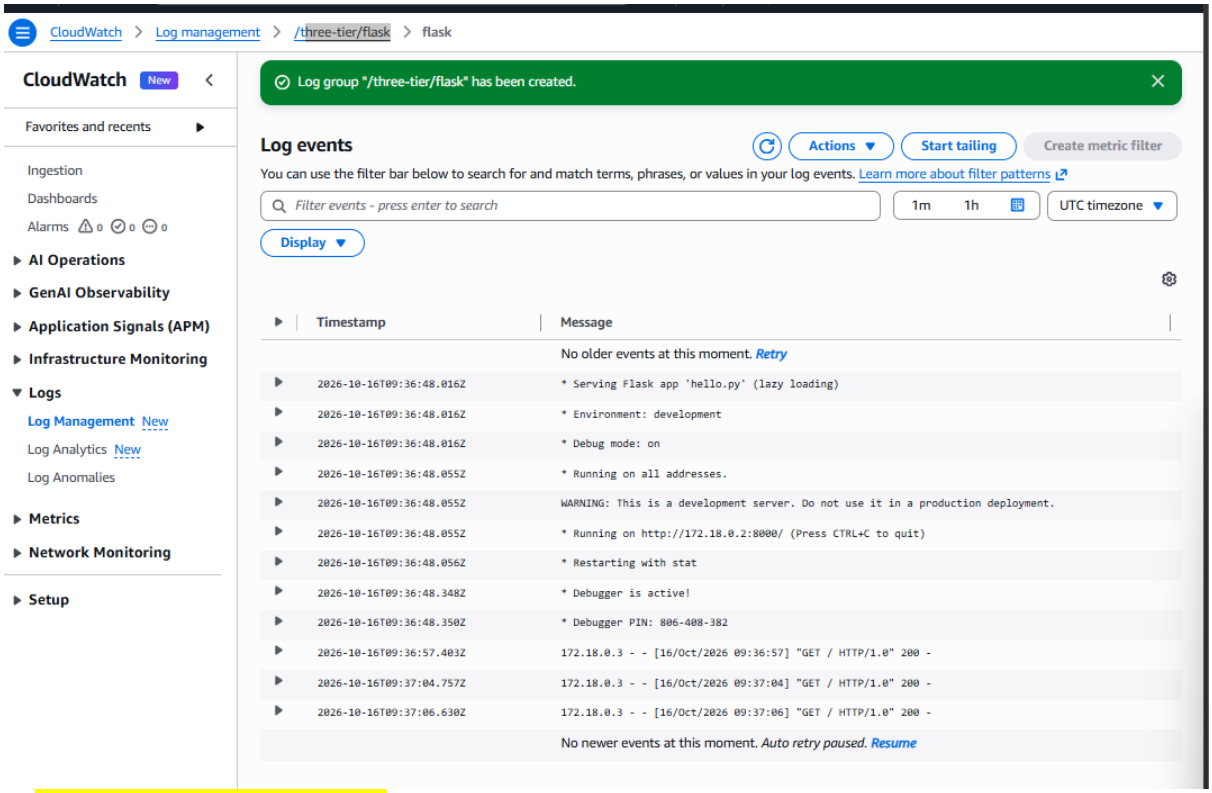
4)navigate to the iam role and select the already created role and choose create inline policy and paste this code

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogStream",
        "logs:PutLogEvents",
        "logs:DescribeLogStreams"
      ],
      "Resource": [
        "arn:aws:logs:ap-south-1: 926583574835:log-group:/three-tier/nginx:*",
        "arn:aws:logs:ap-south-1: 926583574835:log-group:/three-tier/flask:*"
      ]
    }
  ]
}
```

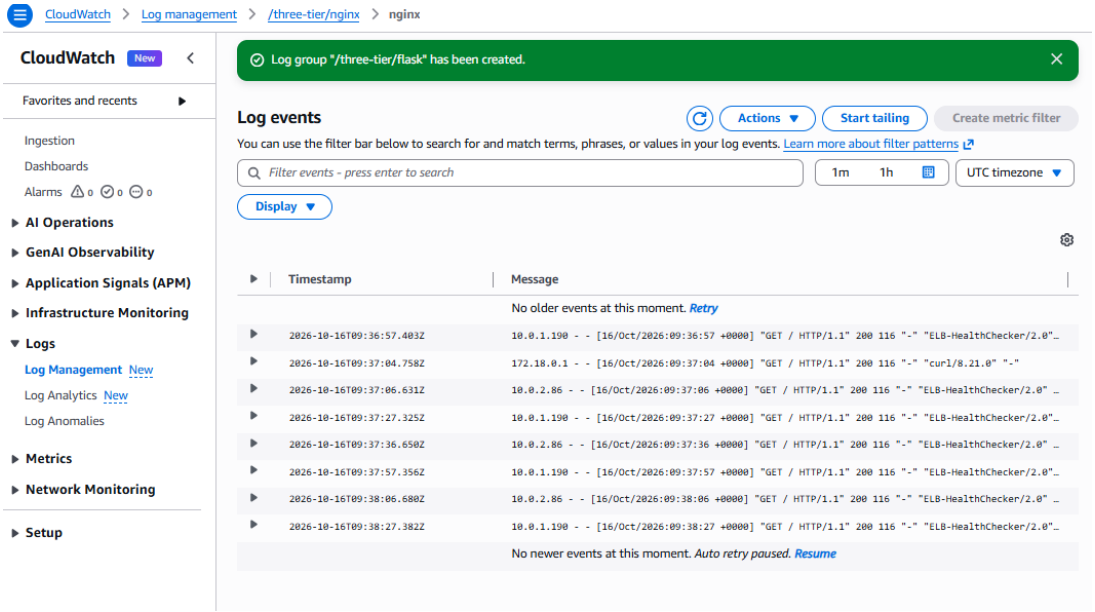
5)give name to the policy click create policy

6)update the iam role to instance.

7)check the /three-tier/flask



8)check the /three-tier/nginx



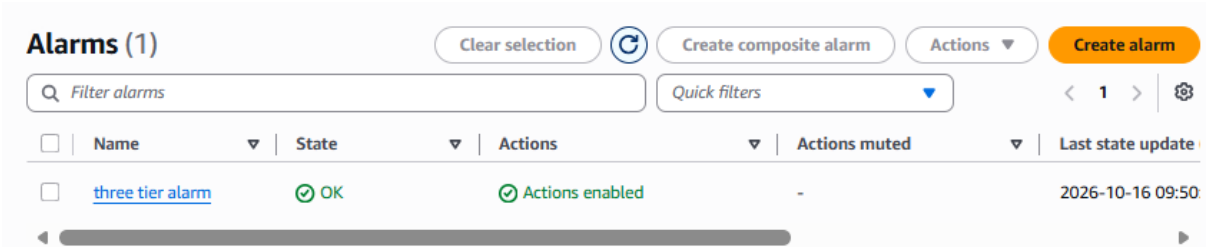
Creation of sns :

1)navigate to the service and give the topic name and add a subscription mail.

Creation of alarm:

1)in cloudwatch add the alarm specify the metrics and add the sns topic.

2)stop the container



SNS NOTIFICATION WHEN THE CONTAINER STOPPED DELIBERATLY:

